

Day 18 Evidence Workbooklet

Python Seminar Synthesis: Plan, Trace, Debug, Explain, Support

No new Python content: integrate planning, tracing, testing, debugging, explanation, and support routing

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/44		Mastery check	secure / developing / needs support	

Concrete engineering situation

Exam 2 is complete. Day 18 does not add a new Python feature. It asks students to combine the bridge evidence routines: plan before code, trace state, test/debug from expected-versus-actual evidence, explain the result, and name a support route.

Engineering task	Turn repeated and stored checkout evidence into a complete, explainable evidence routine.
Computational move	Combine loop/list trace, mutation state, debug/test evidence, and explanatory writing.
Python focus	Review only: for/range, while termination, break/continue, nested loops, list state, indexed mutation, tests, and explanation.
Evidence to keep	problem plan, state ledger, list state after each pass, expected output, actual output, bug cause, minimal fix, protective test, support route.

Day 18 working rules

1. No new Python feature is introduced today.
2. Start with a plan and state variables before code.
3. Trace each pass before trusting the final output.
4. Debug with expected, actual, likely cause, minimal fix, and protective test.
5. End by naming the support route you would use next.

Point map Manage your evidence

blueprint

Points	Section	Evidence focus
1	Inventory the evidence routines you already know	synthesis, inventory
2	Plan before code	plan, state
3	Trace state pass by pass	trace, mutation
4	Debug from expected and actual evidence	debug, test
5	Explain the evidence to another student	explain, transfer
6	Support route for final synthesis	support, reflect

1 Inventory the evidence routines you already know

synthesis

inventory

Circle what you can do independently and name the evidence routine that proves it.

Pts.	Prompt	My evidence / response
2	For/range trace routine: what do you record each pass?	
2	While termination routine: what proves the loop stops?	
2	Break/continue routine: what proves skipped versus stopped work?	
2	List-state routine: what do you write after mutation?	
2	Debug/test routine: what are the five evidence pieces?	

2 Plan before code

plan state

Use the integrated Day 18 code as a synthesis case. Before tracing, identify the state.

```
1 charges = [84, 72, 80, 65]
2 fix_count = 0
3 ready_count = 0
4
5 for index in range(len(charges)):
6     if charges[index] < 80:
7         charges[index] = 80
8         fix_count = fix_count + 1
9     if charges[index] >= 80:
10        ready_count = ready_count + 1
11
12 print(charges)
13 print(fix_count)
14 print(ready_count)
```

Pts.	Prompt	My evidence / response
2	What list state exists before the loop?	
2	What state variables are initialized before the loop?	
2	Why does this job need an index loop?	
2	What should the final list represent?	
2	What should the two counts represent?	

Complete a state ledger for the loop. Keep old value, condition, action, new list state, and count values visible.

```
1 charges = [84, 72, 80, 65]
2 fix_count = 0
3 ready_count = 0
4
5 for index in range(len(charges)):
6     if charges[index] < 80:
7         charges[index] = 80
8         fix_count = fix_count + 1
9     if charges[index] >= 80:
10        ready_count = ready_count + 1
11
12 print(charges)
13 print(fix_count)
14 print(ready_count)
```

Pts.	Prompt	My evidence / response
2	Index 0 ledger: old value, action, list state, counts.	
2	Index 1 ledger: old value, action, list state, counts.	
2	Index 2 ledger: old value, action, list state, counts.	
2	Index 3 ledger: old value, action, list state, counts.	
2	Write the final printed values.	

4 Debug from expected and actual evidence

[debug](#) [test](#)

Suppose a student starts the loop at index 1. Use the expected/actual routine rather than guessing.

Pts.	Prompt	My evidence / response
2	Expected final list for the synthesis code.	
2	Actual final list if the loop starts at <code>range(1, len(charges))</code> .	
2	Likely cause of the mismatch.	
2	Minimal fix.	
2	Protective first-item and last-item tests.	

Common mistake to avoid

A skipped first item can hide inside a final count if you do not write the list state.

5 Explain the evidence to another student[explain](#) [transfer](#)

A secure explanation connects code shape to evidence, not just to a final answer.

Pts.	Prompt	My evidence / response
3	Explain why an index loop is appropriate for this code.	
3	Explain why ready_count reaches 4 after mutation.	
2	Explain why fix_count and ready_count answer different questions.	
2	Write one sentence that would help a student who only guessed the final output.	

6 Support route for final synthesis

support reflect

Close Day 18 by making support visible rather than private.

Pts.	Prompt	My evidence / response
4	Name your weakest evidence routine and the exact support move you will use.	
2	Circle your support route: plan / for trace / while termination / control flow / nested loop / list state / mutation / debug-test / explanation.	
2	Write one commitment for Day 19 support work.	

Support route
Day 19 is support and transition planning. It should strengthen existing evidence routines rather than introduce new Python content.

Copyright © 2026 Lance L.A. White. All rights reserved.